# 10 · Asisten AI, Auto Chart/Dashboard & Story

> Baru: Asisten AI juga tersedia sebagai **panel di setiap halaman** (`Ctrl + I`) dengan peran ahli `/analis`, `/panduan`, `/data-engineer`, `/ahli-ui-ux`, `/hw-engineer`. Lihat 23–asisten-ai-peran-ahli.md.

Asisten analitik yang dipasang **di dalam** Superset pada alamat **http://localhost:8088/asisten/** (menu navbar **Asisten AI**).

| Tab | Fungsi |
|---|---|
| **Tanya AI** | Bertanya dengan bahasa biasa → AI mencari dataset, mengambil data, membuat chart/dashboard |
| **Auto Chart & Dashboard** | Pilih dataset → sistem mempelajari data → daftar saran chart beserta skor & alasan → centang yang mau → jadi dashboard |
| **Story** | Susun beberapa dashboard menjadi presentasi berurutan + narasi (bisa ditulis AI) |

## 1. Cara kerja (baca dulu)

```
Browser ──► /asisten/api/... (Flask blueprint di dalam Superset, login & CSRF sama)
              │
              ├─ agen.py    : putaran  model ⇄ alat  (maks. langkah_maks)
              │     └─ llm.py : POST {base_url}/chat/completions  (lokal / API)
              ├─ alat.py    : alat yang boleh dipanggil model
              │     └─ ChartDataCommand  ← berjalan SEBAGAI USER YANG LOGIN
              │                              → hak akses data (RLS) tetap berlaku
              ├─ auto.py    : profil kolom + skor statistik (pandas/numpy)
              └─ story.py   : tabel sps_story (urutan dashboard + narasi)
```

Prinsip keamanan:

1. **AI tidak pernah menulis SQL.** Model hanya memilih dataset, metrik, kolom, filter. Query dibuat Superset → filter wajib hak akses (`akses.yaml`) selalu ikut.
2. **AI hanya melihat dataset yang boleh dilihat pengguna** (`can_access_datasource`).
3. **Privasi per model** (`kirim_data`): model API cloud bawaannya `false` → angka langsung tampil di layar, tidak dikirim ke luar.
4. **Audit**: setiap pertanyaan tercatat di tabel `sps_ai_log` (siapa, model, alat, status, durasi).

## 2. Struktur berkas

```
config/
├── ai.yaml                  # PENGATURAN: daftar model, siapa boleh, batas kerja
└── sps_ai/
    ├── __init__.py          # titik masuk (daftarkan ke Superset)
    ├── pengaturan.py        # baca ai.yaml + API key dari .env
    ├── llm.py               # klien HTTP OpenAI-compatible
    ├── alat.py              # alat: query data, buat chart, buat dashboard
    ├── auto.py              # Auto chart: profil data + skor "machine learning ringan"
    ├── agen.py              # prompt sistem, definisi alat, putaran agent, audit
    ├── story.py             # simpan/ambil story
    ├── tampilan.py          # halaman & API (/asisten/...), menu navbar, izin
    ├── halaman.html         # UI 3 tab
    └── story.html           # pemutar story (layar penuh)
```

**Konteks otomatis** (peta isi aplikasi, halaman yang dibuka, teks layar) dan alat `info_dashboard`, `info_chart`, `data_chart`: lihat 23 §1b. Kode: `config/sps_ai/wawasan.py`.

## 3. Memasang model

**Cara termudah:** panel Asisten AI (`Ctrl + I`) › **Sambungkan AI**. Model lokal terdeteksi otomatis, key cloud diisi di layar itu (terenkripsi, tanpa restart). Panduan lengkap semua jalur: **28**. Bagian di bawah ini untuk tim TI yang mengatur `config/ai.yaml` secara langsung.

Semua model memakai standar **OpenAI-compatible** dan **wajib mendukung tool calling**. Ganti/tambah model cukup di `config/ai.yaml` — **berlaku tanpa restart**.

### 3a. Model lokal (disarankan, data tidak keluar) — Ollama di Windows

Panduan bergambar untuk pemula (Ollama **dan** LM Studio, kebutuhan RAM, uji, masalah umum): **20-model-ai-lokal.md**. Model lokal dipasang sendiri di Windows, bukan lewat Docker.

```
# 1. Unduh & pasang dari https://ollama.com/download (Windows)
# 2. Unduh model (sekali, ±4,7 GB)
ollama pull qwen2.5:7b
# 3. Tes
ollama run qwen2.5:7b "halo"
```

Container menghubungi Ollama lewat `http://host.docker.internal:11434/v1` (sudah di `ai.yaml`). Bila muncul *connection refused*: set variabel lingkungan Windows `OLLAMA_HOST=0.0.0.0`, tutup Ollama dari tray, buka lagi.

Rekomendasi: `qwen2.5:7b` (RAM 8 GB+), `qwen2.5:14b` (lebih pintar, RAM 16 GB+ / GPU). LM Studio sudah ada di `ai.yaml` (*Qwen 2.5 7B — lokal (LM Studio)*, `http://host.docker.internal:1234/v1`). vLLM juga bisa: ganti `base_url`.

### 3b. Model API (cloud)

Sudah terdaftar di `ai.yaml` (muncul otomatis begitu key-nya diisi). Cara membuat key ada di installer halaman **Asisten AI** › pilih layanan › tombol **? Cara membuat key** (membuka halaman pembuatannya).

| Layanan | Biaya | Variabel `.env` | Model bawaan |
|---|---|---|---|
| Google Gemini | gratis (kuota harian) | `AI_GEMINI_API_KEY` | `gemini-flash-latest` |
| Groq | gratis | `AI_GROQ_API_KEY` | `openai/gpt-oss-120b` |
| OpenRouter | gratis (model `:free`) | `AI_OPENROUTER_API_KEY` | `openai/gpt-oss-120b:free` |
| Cerebras | gratis | `AI_CEREBRAS_API_KEY` | `gpt-oss-120b` |
| Mistral | gratis (paket Experiment, verifikasi HP) | `AI_MISTRAL_API_KEY` | `mistral-small-latest` |
| GitHub Models | gratis dengan akun GitHub (token *Models: read*) | `AI_GITHUB_TOKEN` | `openai/gpt-4.1-mini` |
| Grok (xAI) | kredit awal, lalu berbayar | `AI_XAI_API_KEY` | `grok-latest` |
| OpenAI / Anthropic | berbayar | `AI_OPENAI_API_KEY` / `AI_ANTHROPIC_API_KEY` | `gpt-4o-mini` / `claude-sonnet-4-5` |

Paket gratis: kuota terbatas dan pertanyaan bisa dipakai penyedia untuk melatih model, jadi semua model cloud `kirim_data: false` (angka hasil query tidak dikirim). Nama model penyedia gratis sering berganti: bila muncul *model not found*, ganti `model:` di `ai.yaml` dengan nama di halaman model penyedia (berlaku tanpa restart).

1. Isi key di `.env` (atau lewat installer), mis. `AI_GROQ_API_KEY=gsk_...` (key **tidak** ditulis di `ai.yaml`).
2. `docker compose up -d` (agar `.env` terbaca ulang).
3. Model muncul di pilihan. Yang key-nya kosong otomatis disembunyikan.
4. Jaringan kantor dengan SSL inspection → lihat `certs/README.md`.

### 3c. Menambah model lain

```
- id: deepseek                  # unik, tanpa spasi
  nama: "DeepSeek — API"        # tampil di layar
  base_url: https://api.deepseek.com/v1
  model: deepseek-chat
  api_key_env: AI_DEEPSEEK_API_KEY   # lalu isi di .env
  kirim_data: false
```

## 4. Siapa yang boleh memakai

- `peran_boleh` di `ai.yaml` (bawaan `{role_pengguna}` = role dasar semua pengguna, `SPS_ROLE_PENGGUNA` di `.env`) + Admin.
- Izin menu & akses dataset diberikan oleh `scripts\build-dashboards.ps1`.
- Menu **Asisten AI** dan pilihan model hanya menampilkan model yang **aktif** (server model menjawab dalam 2 detik; diperiksa ulang tiap 60 detik). Tanpa model aktif, menu disembunyikan; tab **Story** tetap bisa dibuka lewat menu **Dashboard & Story → Story** (tanpa tombol narasi AI).
- Pengguna role **Gamma** tidak boleh menyimpan chart → AI memberi **tautan Explore** (chart siap dibuka, belum tersimpan). Untuk boleh menyimpan, beri role Alpha per orang.

## 5. Auto Chart & Dashboard — metodenya

Tidak perlu container tambahan: `pandas` + `numpy` sudah ada di image Superset.

1. **Sampel** maks. 5.000 baris (tetap terfilter hak akses pengguna).
2. **Profil kolom**: *ukuran* (angka), *dimensi* (kategori), *waktu* (tanggal/bulan/periode), *abaikan* (ID/NPWP/NIP).
3. **Skor setiap kandidat chart**:

| Wawasan | Metode | Contoh alasan |
|---|---|---|
| Tren | regresi linear ( `np.polyfit` ), $R^2$ | "tren naik kuat ($R^2$ 0,95)" |
| Konsentrasi | indeks Herfindahl, porsi top-3 | "3 teratas = 72% total" |
| Variasi | koefisien variasi | "perbedaan antar kategori besar" |
| Pencilan | z-score > 2,5 | "ada nilai ekstrem: KPP X" |
| Korelasi | Pearson r ≥ 0,7 | "jumlah_transaksi ↔ total sangat berkorelasi (r 0,98)" |
| Segmen | k-means (numpy) | "kategori terbagi 3 kelompok" |

4. Pengguna memilih saran (centang), boleh ganti judul & jenis chart → **Buat chart** / **Buat dashboard**.

Keterbatasan: skor dari sampel, jadi angka di skor bisa sedikit berbeda dari total penuh.

## 6. Story

- Pilihan dashboard hanya yang **selesai**: berisi chart dan diterbitkan (*published*), atau milik sendiri (*draf*).
- Buat di tab **Story**: judul → tambah dashboard (urutan bisa ↑↓) → narasi tiap langkah (ketik sendiri atau tombol **Tulis narasi dengan AI**) → Simpan.
- Putar: `/asisten/story/<id>` — tombol / panah keyboard ←→.
- **Publik** = semua pengguna asisten bisa melihat. Langkah yang dashboard-nya tidak boleh dibuka pengguna tampil 🔒 (hak akses dashboard tetap berlaku).

- Hanya pemilik (atau Admin) yang bisa mengubah/menghapus.

## 7. Pengaturan lain (`ai.yaml`)

| Kunci | Bawaan | Arti |
|---|---|---|
| `bawaan` | `lokal-qwen` | model yang terpilih pertama kali |
| `langkah_maks` | 6 | maks. pemanggilan alat per pertanyaan |
| `baris_maks` | 500 | maks. baris yang ditampilkan |
| `baris_ke_model` | 50 | maks. baris dikirim ke model ( `kirim_data: true` ) |
| `batas_detik_model` | 120 | waktu tunggu jawaban model |

## 8. Log & audit

```
scripts\logs.ps1                          # cari baris "ALDx AI"
```

```
-- docker compose exec metadb psql -U superset -d sps_superset
SELECT waktu, username, model, pertanyaan, alat, status, durasi_ms
FROM sps_ai_log ORDER BY id DESC LIMIT 50;
```

## 9. Masalah umum

| Gejala | Penyebab | Solusi |
|---|---|---|
| Menu **Asisten AI** tidak muncul | Tidak ada model aktif (LM Studio/Ollama mati) atau izin belum dibuat | nyalakan model (20), tunggu ≤ 1 menit, muat ulang; atau `scripts\build-dashboards.ps1` lalu login ulang |
| "Belum ada model AI yang aktif" | Semua model butuh key / `ai.yaml` salah | Pasang Ollama atau isi key di `.env` |
| "Model tidak bisa dihubungi" | Ollama mati / alamat salah | `ollama list` ; cek `OLLAMA_HOST=0.0.0.0` |
| `CERTIFICATE_VERIFY_FAILED` (model API) | SSL inspection kantor | `AI_CA_BUNDLE` — lihat `certs/README.md` |
| Jawaban AI ngawur / tidak memakai alat | Model terlalu kecil / tanpa tool calling | Pakai `qwen2.5:7b` ke atas |
| "Database sumber data tidak bisa dihubungi" | Oracle/ClickHouse tidak terjangkau (VPN) | Sambungkan VPN, coba lagi |
| AI tidak melihat dataset tertentu | Pengguna memang tidak punya akses | Normal (hak akses berlaku) |